OpenStack コミュニティを対象に 逆強化学習を用いたモジュール別レビュー動態予測

橋本 一輝[1,a)]　伊原 彰紀[1,b)]

概要：**TODO**: []OSS レビュアー継続予測では累積特徴量と Random Forest（RF）が主流であり，行動の時系列パターンを直接扱う研究は限られていた．本研究は行動系列から報酬関数を推定する MCE-IRL を OpenStack 32 リポジトリの 4 予測区間（0–3, 3–6, 6–9, 9–12 か月）に適用し，RF と互角の AUC を示すこと，Archetypal Analysis で分解した開発者タイプ間で MCE-IRL と RF が補完的な得意領域を持つことを確認した．両モデル予測の単純平均によるアンサンブルは最良単独モデル比 +2.2 pp の改善を全予測区間で有意に達成した．

1. はじめに

オープンソースソフトウェア (OSS) は，地理的に分散した場所に居住する開発者からの貢献により支えられていることが多く，開発組織から金銭的な利益を得ることなく，個人の知的好奇心や開発技術の学習などを動機として貢献している開発者も多い [1]．Raymond らは，OSS 開発の成功例として Linux を対象とした調査を行い，このような不特定多数の開発者が自発的に参加する開発形態をバザール方式と名付けた [2]．この開発モデルは，世界中の多様な専門知識を持つ開発者の参加を促進することで，高い品質の OSS の開発を実現している．バザール方式による開発は，新たに貢献する開発者がいる一方で，貢献を停止する開発者も多く、プロジェクト開発要員の流動性が極めて高い [3]．その結果，OSS プロジェクトの継続的な保守が困難になることもある [4]．

OSS の運用・保守は，長期間にわたりプロジェクトに貢献する一部の開発者（LTC: Long-Term Contributor）が担っていることが多く，このような LTC が貢献を停止すると，実装方針，検証基準のような文書化されにくい知識の引き継ぎが困難になる．本研究で対象とするコードレビュー作業において，開発者の離脱は，OSS の品質低下や特定モジュールを熟知する開発者の不足につながる [5][6]．したがって，将来的に発生する開発作業の依頼に対し，貢献する開発者を確保できるかを見積もる手法が期待される．

[1] 和歌山大学
Wakayama Uniersity, Sakaedani 930, Wakayama-city 640–8510, Japan
[a)] s276189@wakayama-u.ac.jp
[b)] ihara@wakayama-u.ac.jp

従来研究では，後継開発者の育成や開発知識の引き継ぎを計画的に進めるために，開発者の将来的な活動頻度や活動期間を事前に予測する手法が提案されている．**memo:** [引用３つの対応] **TODO**: [XX らは，] 開発者がプロジェクトへの貢献量（コミットやイシュー対応，等）に基づき，1 年後以降や 3 年後以降に貢献しているか否かを予測する 2 クラス分類モデルを構築している [7][8][9]．当該手法は離脱の有無を端的に予測できる一方で，活動量の時系列的な変動（増減パターン）を捉えられていない．Shade らは，ある特定の時点のコードレビュー依頼（以降，レビュー依頼）に，開発者が貢献するか否かを予測するモデルを構築しているが，当該研究は将来的な活動を予測するものではない．これら従来研究は，将来的に発生する特定モジュールに関する作業依頼に対し，貢献する開発者を確保できているか見積もる手法は提案していない．OSS プロジェクトのリリースサイクルに同期した周期的活動，集中的に取り扱うモジュールの変化，開発者の貢献停止，貢献再開など，時間的な変動を捉えることが必要となる．

本研究は，開発者への依頼と，開発者が依頼に対して貢献したか否かを明確に観測できるレビュー活動を題材に，将来の開発者のレビュー動態（特定のモジュールに関するレビュー依頼に貢献するか否かの時間経過順の変化）を予測する強化学習モデルを提案する．具体的には，ある任意の時点で活動している開発者が，その先 1 年間を等期間に 4 分割し，それぞれの期間で特定のモジュールのレビュー依頼に貢献するか否かを予測するために，時系列符号化モデル MCE-IRL を用いる．

TODO: [貢献はあとで復活させるかもしれないけど，一旦コメントアウト]

本論文の構成は以下の通りである．2 章で関連研究を整理し，本研究の位置付けを示す．3 章で提案手法である MCE-IRL モデルの構築方法，4 章で評価方法について述べる．5 章でケーススタディを行い，6 章で結果を考察する．7 章で妥当性への脅威を整理し，8 章で結論と今後の課題を述べる．

2. 関連研究

2.1 OSS プロジェクトに貢献する開発者の継続予測

OSS 開発者の継続性に関して多面的に調査されてきた．Iaffaldano らは一時的な活動休止の動機を分析し，私生活の出来事や興味の喪失といった個人要因と，コミュニティ内の対立やプロジェクト方針の変化といった組織要因の両面から開発者が貢献を停止することを示している [10]．Calefato らはコア開発者の休止の頻度と復帰確率を算出している [11]．Vasilescu らは性別および在籍期間の多様性が離脱率に与える影響を分析し，性別多様性は離脱率を低下させる一方で在籍期間多様性は離脱率を増加させることを示している [12]．Lin らは survival 分析を用いて，早期参加・ファイル修正中心・コーディング中心の活動パターンを持つ開発者ほど活動継続期間が長いことを明らかにしている [13]．このように，開発者は多様な理由で貢献を停止することが多くの研究で明らかにされており，事前に開発者の継続性を予測する手法の確立が期待される．

開発者の継続予測モデルを提案する従来研究では，未来の単一時点以降に開発に貢献するか否かの 2 クラス分類モデルを提案している．Zhou と Mockus は，OSS プロジェクトへの新規に参加して 1 か月間の累積活動量から 1 年後以降も継続して貢献するか否かを予測するモデルを構築した [7]．Bao らは GitHub でアカウントを持つ開発者を対象に 1 年後，2 年後，3 年後以降に同じプロジェクトに貢献しているか否かを予測するモデルを構築している [8]．Constantinou と Mens は，RubyGems/npm エコシステムで規則性と長期継続の関連を示した [9]．Shade らは，コードレビュー作業の依頼に，貢献する開発者を推薦する手法を提案しているが，レビュアーが将来も継続して活動するかは明らかにしていない．

これら従来研究では，リポジトリへのコミットやイシューに関する貢献の累積統計量を説明変数とした機械学習モデルを構築し，数年後の特定時点以降の活動の有無を予測するモデルを構築している．しかし，活動量の時系列的な変動（増減パターン）を捉える手法ではなく，将来のレビュー動態を数ヶ月単位で予測する手法ではない．

3. 逆強化学習を用いたモジュール別レビュー動態予測モデル

本研究は，特定のモジュールに対して特定の開発者がレビューに貢献するか否かを予測する（レビュー動態予測）ために，ある任意の時点で活動している開発者の時間経過順の行動を捉えるモデルを構築する．なお，本研究ではモジュールをディレクトリ単位で定義し，以降ではディレクトリと表記する．具体的には，最大因果エントロピー (MCE: Maximum Causal Entropy) と逆強化学習 (IRL: Inverse Reinforcement Learning) を組み合わせた MCE-IRL をレビュー動態予測に適用した手法を提案し，累積統計量に基づく Random Forest (RF) と比較する．通常の IRL は観察された行動の尤度を最大化する報酬関数を探索するが，観察されていない行動に対しては確率が 0 に近くなりやすく，開発者の多様な行動を適切に表現できない．そこで本研究では，観察された行動の特徴量を制約としつつ，未観測の行動の発生確率を 0 としない MCE を用いる．本手法により，過去に観察されていない行動に対しても有限の確率を割り当て，開発者行動の多様性を自然に表現した上で報酬関数を推定することが可能となる．具体的な計算式は 3.3.1 項で述べる．図 1 は訓練期間・予測期間，図は 2 は時間分割方法とラベルの付け方を示す．両図は本章で適宜参照する．

3.1 強化学習・逆強化学習の基礎概念

強化学習 (RL: Reinforcement Learning) は，エージェント（本研究では開発者）が環境に適応しながら将来にわたって得られる報酬の合計（累積報酬）の期待値を最大化する行動選択規則（以後，方策）を学習する枠組みであり，マルコフ決定過程に基づいて定式化される．逆強化学習 (IRL) は，RL を基本とする方法であるため，まずは RL について説明する．

RL は状態 s，行動 a，報酬関数 $r(s,a)$，方策 $\pi(a|s)$ の 4 つの要素から構成される．状態はエージェントが観測する環境の情報，行動はエージェントが選択する操作，報酬関数は状態・行動の組に対して行動の良し悪しを数値で示す．方策は状態に対する行動の確率分布をそれぞれ表す．例えば，Liu らはバグトリアージ問題に RL を適用し，バグレポートの情報を状態，開発者の選択を行動，割り当て結果の正否を報酬，どの開発者を選ぶかの行動選択規則を方策として定義している **TODO**: [引用]．

通常，RL は報酬関数が既知であることを前提とするが，本研究が対象とする開発者行動においては，開発者がレビュー依頼に対して貢献するか否かを選択する際の判断基準を観測できない．したがって，本研究では報酬関数を行動履歴から逆算する IRL を用いる．IRL は，エージェントの状態と行動の時系列記録である行動軌跡 τ を観察し，その行動を説明する報酬関数を逆算する枠組みである．報酬関数 $r_\theta(s,a)$ は n 次元の特徴量ベクトル $\phi(s,a)$ と学習により推定されるパラメータ（重みベクトル）θ の重み付き和 $r_\theta(s,a) = \theta^\top \phi(s,a)$ として表現され，一つの数値としてその状態・行動の組の良し悪しを評価する．なお，θ の

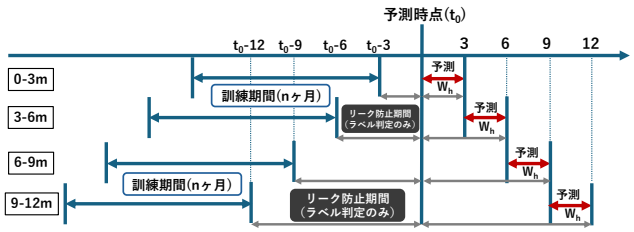


図 1: 各モデルの時間分割方法

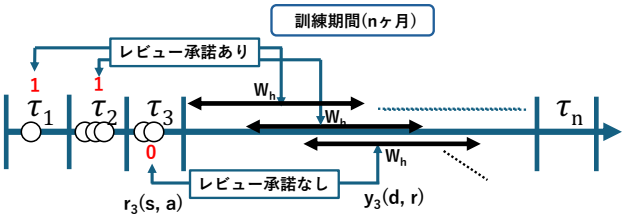


図 2: ラベル付の方法 (例：0-3m モデル)

具体的な決定方法および推定した報酬関数から方策を導く方法については 3.3.1 項で述べる．

本研究では，開発者をエージェント，開発者が観測するレビュー活動の履歴を状態，特定のディレクトリのレビュー依頼に対する承諾・拒否を行動として定義する．特徴量は開発者状態，ディレクトリ親和度，活動特徴の 3 カテゴリ 37 変数（詳細は 3.2,1 項の表 1 に示す）で構成し，IRL により過去のレビュー行動履歴からレビュアーが暗黙的に最適化している判断基準を報酬関数として推定する．

3.2 特徴量・行動軌跡・ラベルの定義

本研究では，開発者 d とディレクトリ r の組 (d, r) を予測単位とし，図 1 に示すスナップショット時刻 t_0 における開発者の状態から，評価開始時点 $h \in \{0, 3, 6, 9$ か月後 $\}$ から評価期間 W_h（本研究では 3 ヶ月間）の内に受けたレビュー依頼に対して貢献するか否かを予測する．

3.2.1 特徴量

状態 s は，表 1 に示す通り，開発者状態（26 次元），ディレクトリ親和度（6 次元），活動特徴（5 次元）の計 37 次元から構成し，両モデルで共通して使用する．

3.2.2 行動軌跡

時刻 t における 1 か月分の記録を $\tau_t = (s_t, \phi_t(s_t, a_t), y_t)$ と定義する．訓練軌跡 $\mathcal{T}$ は図 2 に示すように，各月末を基準点として τ_t をを時間経過順に集約することで，

表 1: 特徴量リスト

カテゴリ	次元数	特徴量（抜粋）
開発者状態	26	累積レビュー数・受諾率・直近性・揺らぎ指標等
ディレクトリ親和度	6	過去レビュー数・直近性・受諾率・所有者重なり等
活動特徴	5	活動の強度（レビュー行数，ファイル数）・応答速度・誰とレビューしたか

$\mathcal{T} = (\tau_1, \tau_2, \ldots, \tau_n)$ として訓練期間 n か月の時系列として構成される．これにより，開発者の行動変化を時間経過順に捉えることが可能となる．なお，行動軌跡は MCE-IRL のみに使用し，RF では使用しない．

3.2.3 ラベル

予測モデルの学習データのラベルとして，レビュー依頼に対して貢献するか否かのラベル $y_h(d, r)$ を図 2 および式 (1) に示す通り定義する．

$$y_h(d,r) = \begin{cases} +1 & W_h\text{内に依頼あり \& 1 件以上承諾} \\ 0 & W_h\text{内に依頼あり \& 全拒否} \\ 0^* & W_h\text{内に依頼なし} \end{cases} \tag{1}$$

0^* は弱負例として重み 0.5（正例・強負例は重み 1.0）を付与する．これは PU 学習の考え方を踏襲し，依頼がない状態を強い負例として扱うと評価期間後半に貢献を再開する開発者を誤学習するため，判断できないケースを小さい重みで保持する [14]．学習に用いるラベルは月単位で，W_h 内のレビュー依頼に対する承諾の有無に基づき付与する．

3.3 モデル構築

本節では，3.2 節で定義した特徴量およびラベルを用いて構築する 2 つのモデルを説明する．まず，本研究で提案する時系列情報を用いて報酬関数を推定する MCE-IRL について述べる．続いて，従来手法にも頻繁に用いられる累積統計量から分類する Random Forest (RF) モデルについて述べる．

3.3.1 提案手法：MCE-IRL に基づくレビュー動態予測モデル

本節では，MCE-IRL のモデル構築手法として，報酬関数の定式化と LSTM を用いた特徴量ベクトルの計算，および開発者が貢献する確率の推定について述べる．

報酬関数 $r_\theta(s, a) = \theta^\top \phi(s, a)$ を決定するために，観測された行動軌跡を説明する報酬関数のうち，方策のエントロピーが最大となるものを求める．ここで，過去のレビュー行動履歴の経験分布を $\mathcal{D}$ とすると，この問題は，過去のレビュー行動履歴から得られる報酬の期待値を最大化しつつ，方策の多様性を保つ次の最適化問題として定式化できる [15]．

$$\max_\theta \mathbb{E}_{\mathcal{T}\sim\mathcal{D}}\left[\sum_t r_\theta(s_t, a_t)\right] - \log Z(\theta) \tag{2}$$

ここで，$\mathbb{E}_{\mathcal{T}\sim\mathcal{D}}[\cdot]$ は過去のレビュー行動履歴 $\mathcal{D}$ から得られた行動軌跡 $\mathcal{T}$ に対する期待値を表す．また，$Z(\theta)$ は方策の確率分布の合計が 1 になるよう調整する正規化定数であり，$-\log Z(\theta)$ は方策のエントロピーを最大化する．エントロピーを最大化することで，未観測の行動に対しても

有限の確率が割り当てることができる..

式 (2) を解くと，方策は $\pi_\theta(a|s) \propto \exp(r_\theta(s,a))$ の形，すなわち softmax 関数として表現される．softmax 関数は，貢献する確率として $[0,1]$ で出力する．学習パラメータ θ の更新には最適化アルゴリズムである Adam **TODO**: [要引用，コメントアウトしてる] を用い，cross-entropy を損失関数として最小化する（学習率 $= 3 \times 10^{-4}$，エポック数 $= 150$，乱数シード $= 42$）．なお，検証データの損失が 15 エポック改善しない場合に学習を早期終了する．

3.4 比較手法：Random Forest を用いたレビュー貢献予測モデル

本研究では，比較対象として従来研究で使用されることの多い Random Forest (RF) を用いたモデルを構築する．RF は複数の決定木を組み合わせた学習手法であり，各決定木の予測結果の多数決により最終的な予測を行う．本研究では，3.2 節で定義した 37 次元の特徴量を平均・分散で正規化した後，RF で学習する [16]．貢献する確率は $[0,1]$ で出力する．学習設定には，決定木の数 $= 100$，乱数シード $= 42$ を用いる．

4. 評価方法

本章では，MCE-IRL と RF の予測精度を比較するための評価方法を述べる．具体的には，比較対象，評価指標と統計検定，および **TODO**: [XX] を明らかにするために原型分析を行うため，その分析方法を述べる．

4.1 評価指標と統計検定

本研究の予測単位は開発者-ディレクトリのペア (d,r) であり，評価もこのペアを基本単位（ペア単位）とする．ただし，積極的に活動する開発者が多くのディレクトリに関わっている場合，その開発者の行動結果が評価に大きく影響するため，開発者ごとに集計した単位（開発者単位）の評価結果も示す．

予測精度の評価指標として，ペア単位および開発者単位の両方に AUC-ROC を用いる．AUC-ROC は，モデルが貢献する開発者と貢献しない開発者をどれだけ正しく識別できるかを $[0,1]$ の数値で表す指標であり，1 に近いほど予測精度が高いことを示す．

MCE-IRL と RF の予測精度の差を統計的に検定するために，開発者単位の AUC-ROC に対して Wilcoxon 符号順位検定（両側）を用いる．効果量は rank-biserial r で示す．

4.2 モデルが予測できるペア・開発者の分析方法

本研究では，MCE-IRL と RF のそれぞれが予測可能な開発者の特徴を明らかにするために原型分析を行う．原型分析は，データ中の極端なパターン（原型）を複数用意し，分析対象のデータ点がどの原型に近いかを表現する手法である [17].

本研究では，予測実験において開発者-ディレクトリのペア (d,r) で捉えるペア単位と，開発者ごとに集計した開発者単位と，区別したように，原型分析においてもペア単位と開発者単位で分析する．

原型分析で対象とする原型数は，原型数を増やしても平均残差平方和の改善が小さくなる点に基づき選択する．なお，原型分析は予測には用いず，MCE-IRL と RF の両モデルが予測可能な開発者の特徴を明らかにする手法として位置付け，RQ2 で調査する．

5. ケーススタディ

5.1 実験設定

対象リポジトリ：評価対象は OpenStack の公式サービスリポジトリ 33 件のうち，分析対象期間にレビュー履歴が存在しなかった sunbeam を除く 32 リポジトリとする．

データ取得と前処理：Gerrit では任意のコードレビュー依頼 1 件につき 1 つのチケットが作成され，レビュアーの指定，議論，ソースコードの承認などが管理される．各チケットにはレビューを依頼した開発者を列挙した reviewers フィールドがあり，本研究ではこのフィールドから依頼された開発者を取得する．ただし，reviewers フィールドに登録されていないが，レビューに貢献した開発者も存在するため，reviewers フィールドの開発者とレビューに貢献した開発者の和集合をレビュー依頼の対象とする．なお，依頼したレビュアーのメールアドレスから bot アカウント（CI/CD 等）を対象外とするために，**TODO**: [XX] とのパターンマッチで自動的に特定する．また，レビュー依頼に対して，開発者が貢献した履歴は，Gerrit REST API[*1]（2024-10 取得）の`/changes/`エンドポイントから，レビュー依頼した開発者がレビュー対象のソースコードに対して承認，要修正，コメントのいずれかの応答を行ったか否かを 2 値ラベルとして取得する．タイムスタンプには各 change の作成時刻 (created) を用いる．

時間分割：評価対象の開始時点 t_0 は，2022 年 01 月 01 日 00 時 00 分 00 秒とし，訓練期間は 3 年間，評価期間は 1 年間とする．なお，t_0 を単一時点に固定しているため，異なる時点での再現性は未検証であり，妥当性への脅威として 7 章で議論する．

標本サイズ：訓練期間と評価期間の両方で観測される (d,r) ペア $n = 4,534$（436 開発者）を分析対象とする．また，原型分析は t_0 直前 6 か月にレビュー・PR 履歴がある 289 開発者のペア $n = 3,747$ に限定する．

5.2 予測モデルの評価結果

本節の主指標はペア単位 AUC-ROC であり，主結果は

[*1] `https://review.opendev.org/`

表 2: 開発者-ディレクトリペア単位 AUC（$n = 4{,}534$）**TODO**: [差し替えと文章の修正]

予測区間	IRL	RF	IRL−RF (pp)
0–3 か月	0.779	0.792	−1.4
3–6 か月	0.770	0.804	−3.4
6–9 か月	0.774	0.789	−1.5
9–12 か月	0.779	0.793	−1.3
4 区間平均	**0.775**	**0.794**	**−1.9**

表 3: 開発者単位 AUC（$N = xxx$）**TODO**: [差し替えと文章の修正]

予測区間	IRL	RF	IRL−RF (pp)
0–3 か月	XXXXX	XXXXX	*XXXX*
3–6 か月	XXXXX	XXXXX	*XXXX*
6–9 か月	XXXXX	XXXXX	*XXXX*
9–12 か月	XXXXX	XXXXX	*XXXX*
4 区間平均	**XXXXX**	**XXXXX**	**XXXX**

$n = 4{,}534$ ペアから算出する．表 2 と表 3 に，それぞれ開発者-ディレクトリペア単位と開発者単位の MCE-IRL と RF の AUC-ROC の結果を示す．各表では 4 予測区間を区別して示す．ペア単位における両モデルの差は **TODO**: [±4 pp] 以内に収まり，MCE-IRL は RF とほぼ互角の精度であった．一方，開発者単位における両モデルの差は **TODO**: [±4 pp でありほげほげほげ]．次節で両モデルが予測できる開発者の特徴を分析する．

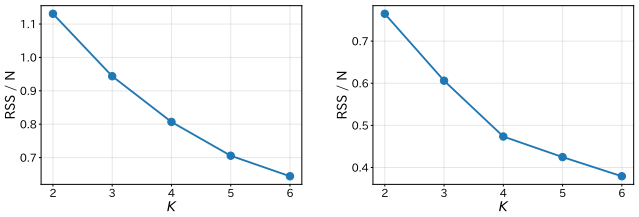
5.3 モデルが予測できるペア・開発者の分析結果

本節では，4 章で述べた原型分析を用いて，MCE-IRL と RF のそれぞれが予測できる開発者および開発者-ディレクトリペアの特徴を分析する．分析対象は t_0 直前 6 か月にレビュー・PR 履歴がある開発者 289 人，およびそれらに関与するペア $n = 3{,}747$ ペアに限定する．原型数 K はいずれも RSS/N の elbow に基づき，$k = 3 \to 4$ で RSS が大きく改善し $k = 5$ 以降は改善幅が小さくなったため，解釈性も考慮して両粒度とも $K = 4$ を採用した．

本節では Archetypal Analysis を 2 つの粒度で実施する．第一に開発者単位で開発者そのものを分類し，第二に開発者-ディレクトリペア単位で両者の関係性を分類する．対象は t_0 直前 6 か月にレビュー・PR 履歴がある開発者（$N = 289$）およびそれらが関与するペア（$n = 3{,}677$）とする．図 3 は，原型数 K と原型数に対する精度改善を示す平均残差平方和を示す．原型数 $k = 3$ から $k = 4$ で平均残差平方和が最も大きく改善し $k = 5$ 以降は改善幅が小さくなったため $K = 4$ を採用する．

5.3.1 ペア単位の分析

ペア単位の原型分析では，開発者とディレクトリの関係性を分類する．特徴量は開発者状態 26 次元とディレクトリ親和度 6 次元の計 32 次元を用いる．図 **??**(a) により $K = 4$ を採用し，表 4 と図 4 は，原型分析によって設定した 4 つの原型の解釈名，および，それぞれの原型の特徴量の分布を示す．表 4 に示す特徴の列は，図 4 の分析結果をまとめている．図 4 の横軸は他の原型の平均との差，縦軸は特徴量名を示す．



(a) ペア単位（$n = 3{,}677$）　(b) 開発者単位（$N = 289$）

図 3: 原型数 K に対する平均残差平方和の推移．

表 5 にペア単位の 4 原型における予測区間別の AUC 差（IRL−RF）を示す．P1（ディレクトリ未経験）では 4 区間中 3 区間で MCE-IRL の方が精度が高く，貢献経験が少ない開発者-ディレクトリペアには時系列情報を用いた MCE-IRL を使用するのが良い．一方で，一方，P2（経験あり・低承諾）と P3（経験あり・高承諾）では RF の方が精度が高く，貢献経験が豊富な開発者-ディレクトリペアに対しては累積統計量を用いた RF を使用するのが良い．P4（中程度関与）は，区間によって優位なモデルが混在する．したがって，ディレクトリ経験が薄い領域では MCE-IRL の時系列推論が，経験が厚い領域では RF の累積統計量が有用であることを示した．

5.3.2 開発者単位の分析

開発者単位の原型分析では，開発者を分類する．特徴量は開発者状態 26 次元を用いる．図 **??**(b) により $K = 4$ を採用し，表 7 と図 5 は，原型分析によって設定した 4 つの原型の解釈名，および，それぞれの原型の特徴量の分布を示す．表 5 にペア単位の 4 原型における予測区間別の AUC 差（IRL−RF）を示す．

表 7 に開発者単位の 4 原型における予測区間別の AUC 差（IRL−RF）を示す．A1（低活動の新人）は，全区間で RF の方が精度が高く，活動履歴が薄い開発者に対しては累積統計量を用いた RF を使用するのが良い．一方，A2（レビュー専念型）では全区間で MCE-IRL の方が精度が高く，行動変化を捉える必要がある開発者に対しては時系列情報を用いた MCE-IRL を使用するのが良い．A3（高承諾のベテラン）と A4（常時活動型）では区間によって優位なモデルが混在する．したがって，活動履歴が薄い開発者には RF の累積統計量が，行動変化が見られる開発者には MCE-IRL の時系列推論が有用であることを示した．

表 4: 4 ペア archetype の解釈名（$n = 3{,}677$）

ID	名前	特徴	n
P1	ディレクトリ未経験	他プロジェクトへの貢献経験（cross_proj）を有し，最終活動からの経過時間（path_recency）が短いが，ディレクトリへの過去レビュー数（path_n_rev）は少ない	1,284
P2	経験あり・低承諾	最終活動からの経過時間（path_recency）が長く，承諾率（accept_rate_all 等）が低い	928
P3	経験あり・高承諾	承諾率（accept_rate_all 等）が高い	539
P4	中程度関与	全体承諾率（accept_rate_all）が低く，活動頻度（act_freq）も少ない	926

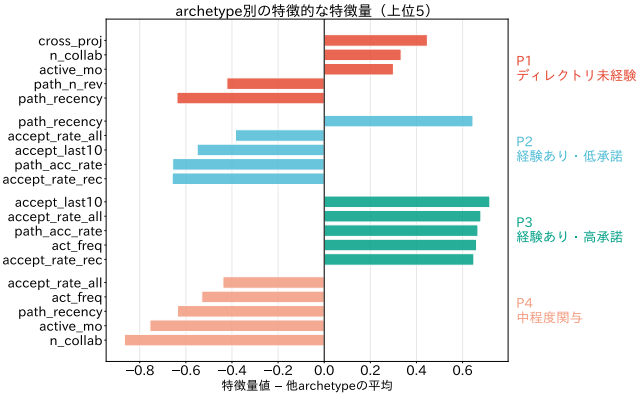


図 4: 4 ペア archetype の特徴的な特徴量

表 5: ペア単位の 4 原型における予測区間別の AUC 差（IRL-RF）（pp，$n = 3{,}677$） **TODO**: [差し替え]

Arch.（n）	0–3m	3–6m	6–9m	9–12m
P1 (1,284)	+5.7	+0.1	+6.9	−1.6
P2 (928)	−7.6	−6.1	−3.3	−4.9
P3 (539)	−8.9	−8.9	+1.1	−1.3
P4 (926)	+4.1	−1.9	−3.9	+2.9

表 6: 4 原型の解釈名（$N = 289$）

ID	名前	特徴	n
A1	低活動の新人	**TODO**: [XX]（same_domain）は高いが，活動頻度（act_freq）や活動月数（active_mo）や **TODO**: [XX]（rev/owner）が低い	59
A2	レビュー専念型	**TODO**: [XX]（rev/owner）は高いが，活動月数（active_mo）や承諾率（accept_rate_all）が低い	69
A3	高承諾のベテラン	直近の承諾率（accept_last10，accept_rate_rec）や活動頻度（act_freq）が高い	63
A4	常時活動型	活動月数（active_mo）が長いが，承諾率（accept_rate_all 等）が低い	98

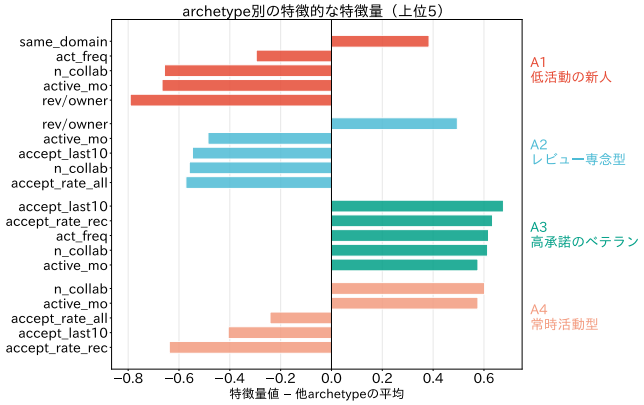


図 5: 4 開発者 archetype の特徴量プロファイル

表 7: 開発者単位の 4 原型における予測区間別の AUC 差（IRL-RF） **TODO**: [差し替え]

Arch.（n）	0–3m	3–6m	6–9m	9–12m
A1 (341)	−11.3	−12.3	−6.9	−5.6
A2 (554)	+1.2	+3.9	+2.1	+5.6
A3 (1,213)	+2.1	−6.1	−0.9	−2.2
A4 (1,639)	−3.3	−4.3	+0.2	−0.4

5.4 追加実験：アンサンブルモデル

5.4.1 概要

5.3 節で確認した MCE-IRL と RF は，予測可能な開発者とディレクトリのペア，開発者が異なるため，モデル間で補完することで高い精度が期待できる．追加実験として両モデルの貢献確率の単純平均をアンサンブルの予測値とし，予測精度の改善を検証する．

5.4.2 アンサンブルモデルの評価結果

表 8 にアンサンブルの結果を示す．全 4 予測区間で 95% 信頼区間が 0 を含まず，**TODO**: [何と比較して] 平均 +2.2 pp の有意な改善が確認できる．

表 10 にペア単位の 4 原型における予測区間別のアンサンブルモデルが最良単独モデルからの改善精度を示す．全ての区間，4 つの原型で正の改善（+0.6〜+13.0 pp）が得られており，特に P1（ディレクトリ未経験）と P4（中程度関与）で改善幅が大きい（最大 +13.0 pp）．これは 5.3 節で確認したように，本研究で提案する MCE-IRL が効果を発する条件において，アンサンブルモデルでも効果が大きい．

TODO: [Wilcox 符号順位検定（両側）はどの結果？]

TODO: [開発者単位の結果は書かないの？]

6. 考察

6.1 なぜ開発者タイプ別の補完性が生じるか

??節で観察された補完性は，開発者 archetype ごとの行動構造に起源を持つと解釈できる．

表 8: MCE-IRL，RF，アンサンブルによるモデルの精度比較結果と，bootstrap による 95%信頼区間． **TODO**: [差し替え]

区間	IRL	RF	アンサンブル	Ens−best (pp)	95% CI
0–3m	0.779	0.792	0.813	+2.1	[+1.4, +2.8]
3–6m	0.770	0.804	0.819	+1.5	[+0.8, +2.2]
6–9m	0.774	0.789	0.816	+2.7	[+2.0, +3.4]
9–12m	0.779	0.793	0.815	+2.3	[+1.4, +3.2]
平均	0.775	0.794	**0.816**	**+2.2**	—

表 9: ペア単位の 4 原型におけるアンサンブルモデルを用いた予測区間別の AUC 差 **TODO**: [差し替え]

Pair Arch. (n)	0–3m	3–6m	6–9m	9–12m
P1 (1,284)	+5.5	+7.3	+2.7	+9.8
P2 (928)	+2.6	+2.4	+1.6	+3.3
P3 (539)	+0.6	+4.1	+8.5	+6.0
P4 (926)	+9.9	+5.5	+5.5	+13.0
ALL (3,677)	+6.4	+4.4	+5.5	+5.7

表 10: 開発者単位の 4 原型におけるアンサンブルモデルを用いた予測区間別の AUC 差 **TODO**: [差し替え]

Developer Arch. (n)	0–3m	3–6m	6–9m	9–12m
A1				
A2				
A3				
A4				
ALL (3,677)	+6.4	+4.4	+5.5	+5.7

A1（低活動の新人）では，履歴量自体が乏しく月次系列が短く疎であるため，LSTM hidden state が安定した選好を符号化することが難しい．一方で累積活動量の少なさそのものが強い負例特徴量となり，分類ルール型の RF が保守的に優位を保つ．結果として 4 予測区間すべてで RF 優位が一貫し，最大 −12.3 pp（3–6 か月）の差が観測される．

A2（レビュー専念型）では，訓練期間の累積プロファイルが「ほぼ拒否」に偏る一方で，評価期間に部分的に承諾方向へ揺らぐ開発者が一定数含まれる．累積統計のみに依存する RF は訓練期間の低い承諾率に引きずられて追従できないが，月次系列を介して状態変化を取り込める MCE-IRL は部分的な行動変化を捉えやすい．これが 4 予測区間を跨ぐ IRL 優位の本質的な機序であり，特に 9–12 か月で +5.6 pp の最大差として顕在化する．

A3（高承諾のベテラン）と A4（常時活動型）では，訓練期間と評価期間で振る舞いが安定しているため，両モデルとも十分な情報を得られ絶対精度が拮抗する．このとき両モデルの誤りパターンが直交するセルでアンサンブルの効果が表れやすく，**??**節の利得分布と整合する．

開発者 archetype とペア archetype で補完性の現れ方が異なる点も注目に値する．ペア archetype では，ディレクトリ経験が薄い領域（P1 ディレクトリ未経験）で時系列推論を介した MCE-IRL が効き，ディレクトリ経験が厚い領域（P2 経験あり・低承諾，P3 経験あり・高承諾）では累積統計に依存する RF が効くという構造が観察された．ディレクトリ親和度を含む pair 単位で見るほうが補完性が鮮明になる事実は，予測単位として開発者-ディレクトリペアを採用した本研究の設計と整合する．

6.2 なぜアンサンブルが効くか

両モデルの予測確率を単純平均する設計が実際に精度向上をもたらす理由は次の 2 点に整理できる．

第一に，両モデルが同じ問題に対して互いに逆方向の誤りを犯すサブグループが存在する．A1 では RF が一貫優位（最大 −12.3 pp），A2 では MCE-IRL が一貫優位（最大 +5.6 pp）であり，符号反転を伴う補完性が 4 予測区間を跨いで安定して観測される．こうした構造下では，固定重み 0.5 の単純平均でも双方の誤りが部分的に相殺され，全体としての識別力が底上げされる．

第二に，ペア archetype × 予測区間の 16 セルすべてで正のアンサンブル gain（+0.6〜+13.0 pp）が得られており，ペア archetype で補完性が鮮明な P1 と P4 で改善幅が特に大きい．これは「補完性の強いセルほどアンサンブルの効きが大きい」という関係を支持し，5.3.1 節の符号反転パターンと**??**節の利得分布が同じ機序から導かれることを示す．

7. 妥当性への脅威

7.1 内的妥当性

主たる懸念は時間方向のラベルリークである．ベースモデルは訓練期間（2019–2022）と評価期間（2022–2023）を時系列で厳密に分離して学習し，特徴量抽出も $[t_0 - W_{\text{train}}, t_0]$ の閉区間に限定しているため，将来情報の参照は設計上排除されている．アンサンブルは固定重み 0.5 の単純平均で学習可能パラメータを持たないため，統合段階で追加のリークは発生しない．Archetypal Analysis は評価集合の特徴量から原型と重み $\boldsymbol{\alpha}$ を計算しているが，補完性の事後解釈にのみ用いておりベースモデルの学習・評価とは独立しているため，主結果の AUC には影響しない．

ハイパーパラメータ選択の影響も懸念となるが，MCE-IRL と RF のパラメータは事前の予備実験で確定し，本論文の最終評価集合に対する再選択は行っていない．アンサンブルは固定重み 0.5 で学習可能パラメータを持たないため，統合段階でのチューニング由来のバイアスは存在しない．

7.2 外的妥当性

評価対象は OpenStack を Gerrit で運用している 32 main repo に限定されている．GitHub の Pull Request ベースのワークフローを採るプロジェクトとは，レビュー単位や承

諾の表現粒度が異なるため，本研究の補完性パターンが他のレビュープラットフォームでも同様に観測されるかは未検証である．ただし，アンサンブルの効きの機序は「累積統計と時系列符号化が互いに直交する誤り方向を持つ」という構造に基づき，この構造はレビューワークフローに強く依存しないと考えられるため，他の OSS においても補完性の符号は反転しうるが何らかの補完性が現れる可能性は期待できる．

スナップショット時刻を t_0 を 2022 年 1 月 1 日に固定しているため，異なる日時での再現は未検証である．ただし，OpenStack は半年ごとに新バージョンをリリースするため，訓練期間（3 年間）および評価期間（1 年間）にはリリース前後の活動量の変動が複数回含まれる．したがって，リリースサイクルに依存した偏りは分析対象期間内で一定程度吸収されており，単一の t_0 であっても分析としての価値は十分あると考える．

7.3 構成的妥当性

本研究では「継続」を「指定された予測区間内にレビュー依頼を受け，1 件以上承諾すること」と操作的に定義した．依頼を確認したが応答しなかった「沈黙のレビュアー」は全拒否と同等に扱われ，依頼を見ていない不在状態とは区別できない．ただし 3.2 節のラベル定義では，依頼自体が発生しなかったペアを弱負例（重み 0.1）として強い負例と分離しており，不在状態を承諾拒否と同等に誤学習するリスクは設計上抑制されている．

8. おわりに

本研究は，特定のモジュールに対して特定の開発者がレビューに貢献するか否かの予測に対し，時系列情報を用いる MCE-IRL と累積統計量を用いる RF のモデルを構築した．ケーススタディとして，OpenStack の公式サービスリポジトリ 32 件を対象に実験を行った．原型分析で分解した開発者のタイプ間では，両方のモデルは予測結果を補完しあうことで精度向上が期待されることを明らかにし，アンサンブルモデルを提案した．その結果，それぞれのモデルの最大値に比べ，平均 +2.2 pp の改善が見られた．今後は，MCE-IRL が推定した報酬関数を環境のフィードバックとして組み込み，開発者の継続性を考慮したレビュー依頼の推薦方策の設計と評価へ展開する．

謝辞 本研究は JSPS 科研費 25K15058 の助成を受けたものです。

参考文献

[1] Ye, Y. and Kishida, K.: Toward an understanding of the motivation of open source software developers, *25th International Conference on Software Engineering, 2003. Proceedings.*, IEEE, pp. 419–429 (2003).

[2] Raymond, E.: The cathedral and the bazaar, *Knowledge, Technology & Policy*, Vol. 12, No. 3, pp. 23–49 (1999).

[3] Howison, J., Inoue, K. and Crowston, K.: Social dynamics of free and open source team communications, *IFIP International Conference on Open Source Systems*, Springer, pp. 319–330 (2006).

[4] Pinto, G., Steinmacher, I. and Gerosa, M. A.: More common than you think: An in-depth study of casual contributors, *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, IEEE, pp. 112–123 (2016).

[5] Bacchelli, A. and Bird, C.: Expectations, outcomes, and challenges of modern code review, *2013 35th International Conference on Software Engineering (ICSE)*, IEEE, pp. 712–721 (2013).

[6] McIntosh, S., Kamei, Y., Adams, B. and Hassan, A. E.: An empirical study of the impact of modern code review practices on software quality, *Empirical Software Engineering*, Vol. 21, No. 5, pp. 2146–2189 (2016).

[7] Zhou, M. and Mockus, A.: What make long term contributors: Willingness and opportunity in OSS community, *2012 34th International Conference on Software Engineering (ICSE)*, IEEE, pp. 518–528 (2012).

[8] Bao, L., Xia, X., Lo, D. and Murphy, G. C.: A large scale study of long-time contributor prediction for GitHub projects, *IEEE Transactions on Software Engineering*, Vol. 47, No. 6, pp. 1277–1298 (2019).

[9] Constantinou, E. and Mens, T.: An empirical comparison of developer retention in the RubyGems and npm software ecosystems, *Innovations in Systems and Software Engineering*, Vol. 13, pp. 101–115 (2017).

[10] Iaffaldano, G., Steinmacher, I., Calefato, F., Gerosa, M. A. and Lanubile, F.: Why do developers take breaks from contributing to OSS projects? A preliminary analysis, *ACM Transactions on Software Engineering and Methodology*, Vol. 32, No. 3, pp. 1–34 (2023).

[11] Calefato, F., Gerosa, M. A., Iaffaldano, G., Lanubile, F. and Steinmacher, I.: Will you come back to contribute? Investigating the inactivity of OSS core developers in GitHub, *Empirical Software Engineering*, Vol. 27, No. 3, p. 76 (2022).

[12] Vasilescu, B., Posnett, D., Ray, B., van den Brand, M. G. J., Serebrenik, A., Devanbu, P. T. and Filkov, V.: Gender and tenure diversity in GitHub teams, *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pp. 3789–3798 (2015).

[13] Lin, B., Robles, G. and Serebrenik, A.: Developer turnover in global, industrial open source projects: Insights from applying survival analysis, *2017 IEEE 12th International Conference on Global Software Engineering (ICGSE)*, IEEE, pp. 66–75 (2017).

[14] Elkan, C. and Noto, K.: Learning classifiers from only positive and unlabeled data, *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 213–220 (2008).

[15] Ziebart, B. D., Maas, A., Bagnell, J. A. and Dey, A. K.: Maximum entropy inverse reinforcement learning, *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438 (2008).

[16] Breiman, L.: Random forests, *Machine Learning*, Vol. 45, No. 1, pp. 5–32 (2001).

[17] Cutler, A. and Breiman, L.: Archetypal analysis, *Technometrics*, Vol. 36, No. 4, pp. 338–347 (1994).